

NETROUTE

Network Packet Routing Simulator

PROJECT REPORT

Data Structures & Algorithms (DSA)- CSL2020

Course Instructor- Debasis Das

Indian Institute of Technology Jodhpur

TEAM MEMBERS

Name	Roll Number
Kolapkar Vipul Kishor	B24CI1021
Kartik Jain	B24CI1020
Saipriya	B24CI1047
Patil Rushikesh Vasant	B24CH1027

1. Abstract

NetRoute is an interactive network packet routing simulator designed to make complex graph algorithms feel real and intuitive. Instead of just studying theory, users can actually see how routing decisions play out across a global network.

The system is built as a full-stack application. A high-performance C++ engine handles the heavy algorithmic computations, while a Node.js and Express backend manages communication and processing. On the frontend, HTML, JavaScript, and D3.js bring everything to life with smooth, real-time visualizations.

NetRoute runs on a global network of 30 nodes and 51 edges, representing cities across six continents. Users can explore four key graph algorithms in action: Dijkstra's Shortest Path (for a single route), Dijkstra across all nodes, Prim's Minimum Spanning Tree, and Kruskal's Minimum Spanning Tree. To make it more realistic, routing decisions can be based on either latency (in milliseconds) or cost (in USD).

One of the most engaging features is its dynamic nature. Users can add or remove nodes and edges directly from the browser and instantly see how the network adapts.

More than just a learning tool, NetRoute reflects how real-world networks operate. The algorithms behind it are the same ones that power modern routing systems like OSPF and BGP, as well as large-scale fiber network planning used by telecom companies around the world.

2. Introduction

Modern computer networks route billions of packets every second across geographically distributed, complex topologies. The efficiency and correctness of this process depends entirely on the graph algorithms that compute optimal paths. Despite their critical importance, these algorithms are rarely presented in a visually interactive format that makes their behavior intuitive.

NetRoute addresses this by providing a visually rich, computationally accurate simulation environment where users can:

- Find the shortest path between any two nodes using Dijkstra's algorithm, filtered by latency or cost
- Compute all-pair shortest distances from any source node across the full 30-node topology
- Visualize the Minimum Spanning Tree using both Prim's and Kruskal's algorithms simultaneously
- Compare MST vs Dijkstra paths side-by-side to understand the difference between minimum backbone and minimum route
- Dynamically add/delete nodes and edges with live re-rendering of the graph
- Inspect the network topology via a searchable node directory

The project architecture is a clean three-tier system: C++ handles all algorithmic computation, Node.js orchestrates subprocess execution and serves the API, and the browser renders the live interactive graph using D3.js force simulation.

3. Application Interface

NetRoute features a modern dark-themed three-panel layout designed for clarity, with instant visual feedback for every algorithm run. The legend distinguishes three edge types: Cable (grey), MST Edge (green), and Path (cyan).

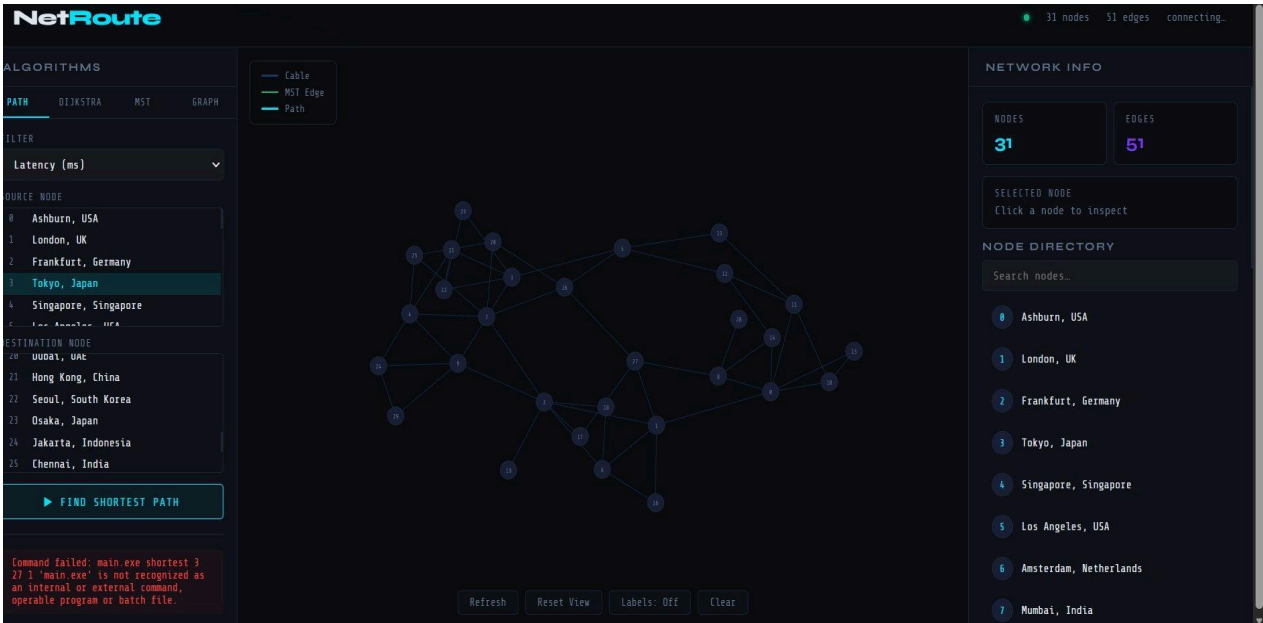


Figure 1: NetRoute main interface — 30-node global topology with server online status

3.1 Panel Layout

Panel	Location	Contents
Algorithm Control	Left sidebar	PATH / DIJKSTRA / MST / GRAPH tabs; filter selector (Latency/Cost); source & destination pickers; action buttons
Graph Viewport	Centre canvas	D3.js force-directed graph; interactive pan/zoom; colour-coded edges; real-time path/MST highlighting
Network Info	Right sidebar	Live node/edge counter; selected node inspector; searchable node directory with all 30 global cities

4. Features & Algorithm Deep Dive

4.1 PATH — Dijkstra's Shortest Path (Single Pair)

The PATH tab finds the optimal route between two user-selected nodes. The user picks a source (e.g. Frankfurt, Germany) and a destination (e.g. Singapore, Singapore), selects a filter (Latency or Cost), and

clicks FIND SHORTEST PATH. The C++ backend executes Dijkstra's algorithm and returns the path, hop count, and total metric value, which is then highlighted in cyan on the canvas.

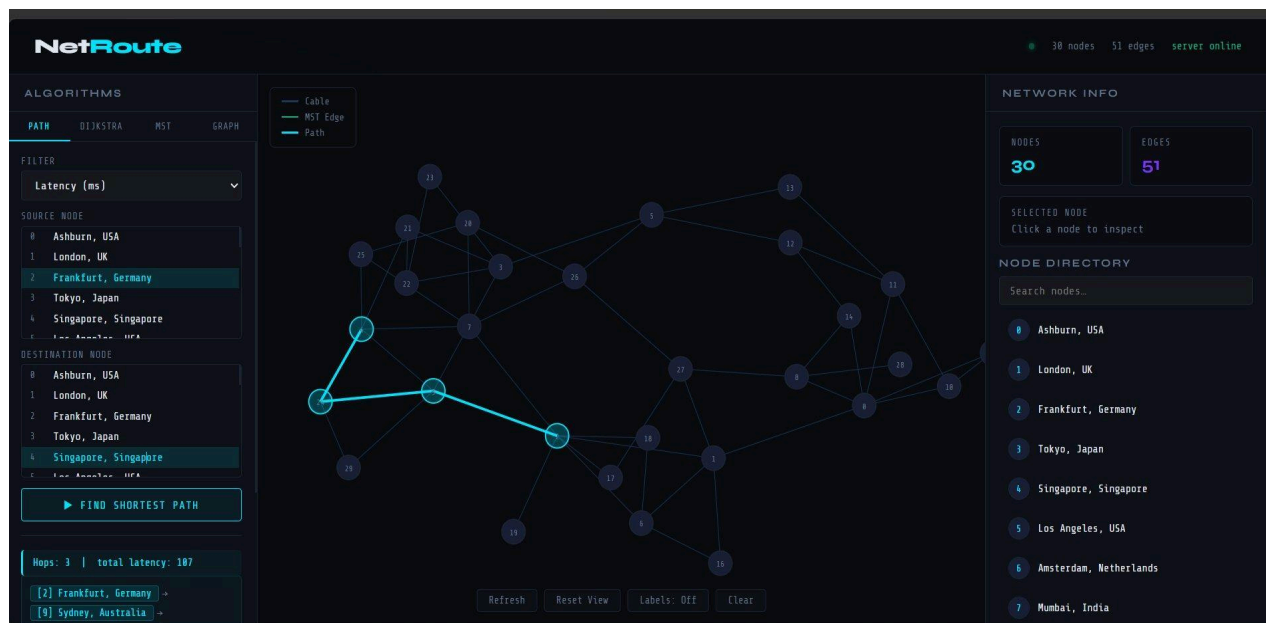


Figure 2: PATH result — Frankfurt, Germany → Singapore, Singapore via 3 hops, total latency 107 ms (cyan path highlighted)

Algorithm Approach:

- The graph is represented as an adjacency list with weighted edges (latency or cost). A min-priority queue selects the node with the smallest distance, and edge relaxation updates distances for unvisited neighbors. The shortest path is reconstructed using a previous-node array.
- The output includes hop count, total weight, and the ordered path. The algorithm runs in $O((V + E) \log V)$ time with $O(V + E)$ space, making it efficient for large networks.

4.2 DIJKSTRA — All-Nodes Shortest Path

The DIJKSTRA tab computes the shortest distance from one source node to every other node in the network simultaneously. The result is a complete distance table — essentially a full routing table for the selected source — displayed in the left panel sorted by distance.



Figure 3: Dijkstra All — distances from Sydney, Australia to all 30 nodes (sorted by latency in ms)

Key Observations from Figure 3:

- Amsterdam, Netherlands: 39 ms — closest hub to Sydney in this topology
- London, UK: 46 ms | Singapore: 75 ms | Mumbai: 133 ms
- São Paulo, Brazil: 220 ms — longest path, consistent with geographic distance

4.3 MST — Prim's Algorithm

The MST tab provides two buttons: PRIM'S and KRUSKAL'S. Prim's algorithm grows the minimum spanning tree from a seed node by greedily selecting the minimum-weight edge connecting the current tree to any unvisited node. The result is rendered as green MST edges on the graph, and the complete MST edge list with weights is shown in the left panel.



Figure 4: Prim's MST — total MST latency 726 ms with all 29 MST edges listed (e.g. [9]↔[29] weight: 10)

Data Structures Used:

- Adjacency list representation of the graph
- Min-priority queue for greedy minimum-edge selection
- Visited array to track which nodes are already in the MST
- Time Complexity: $O(E \log V)$ | Produces exactly $V-1$ edges

4.4 MST — Kruskal's Algorithm

Kruskal's approach sorts all 51 edges globally by weight and greedily adds each edge if it does not form a cycle, detected using a Union-Find (Disjoint Set Union) data structure with path compression and union by rank. Both algorithms produce the same MST total latency of 726 ms, confirming correctness.



Figure 5: Kruskal's MST — total latency 726 ms, edges sorted globally (e.g. [0]↔[10] weight: 5, [2]↔[6] weight: 7)

Data Structures Used:

- Edge list sorted in $O(E \log E)$ time
- Union-Find with path compression: near $O(\alpha(V))$ per operation
- Both Prim's and Kruskal's produce identical MST — verified by equal total weight (726 ms)
- Kruskal's preferred for sparse graphs; Prim's preferred for dense graphs

4.5 MST vs Dijkstra Path Comparison

A standout feature of NetRoute is the COMPARE BOTH button in the MST tab. This runs both the MST backbone path and Dijkstra's shortest path for the same source-destination pair simultaneously, displaying them side-by-side to illustrate a critical concept: the MST minimises the total network cost, while Dijkstra minimises the individual route.

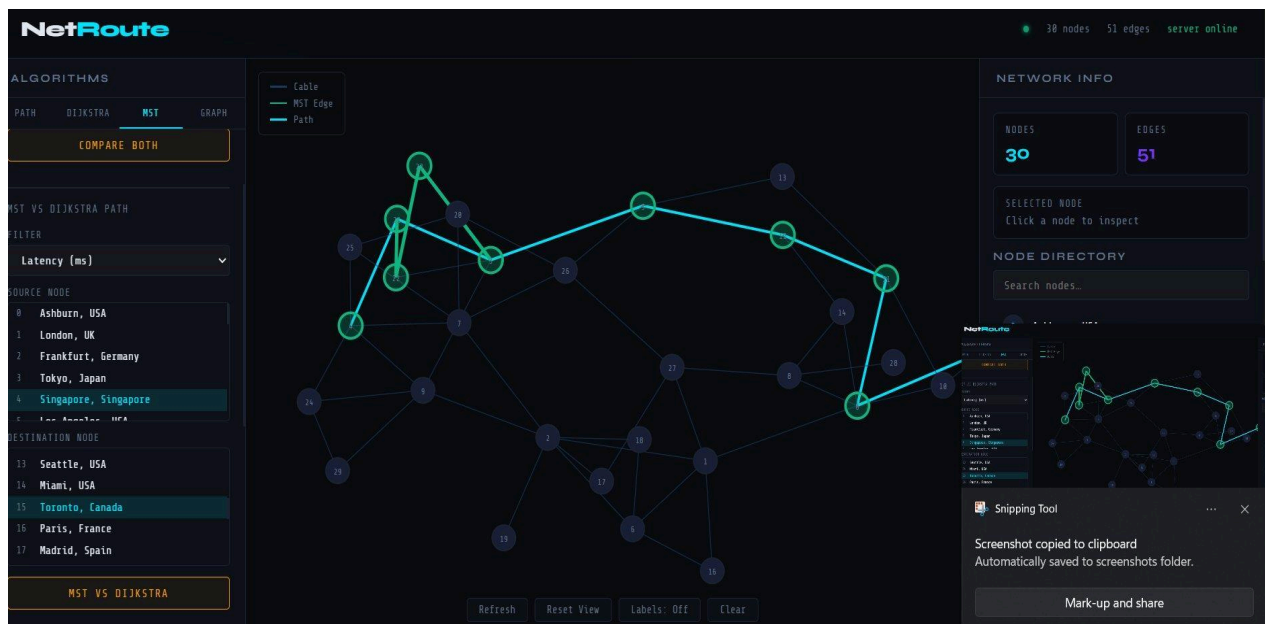


Figure 6: MST vs Dijkstra comparison — Singapore → Toronto; Full Graph: 7 hops / 182 ms vs MST Graph: 9 hops / 200 ms

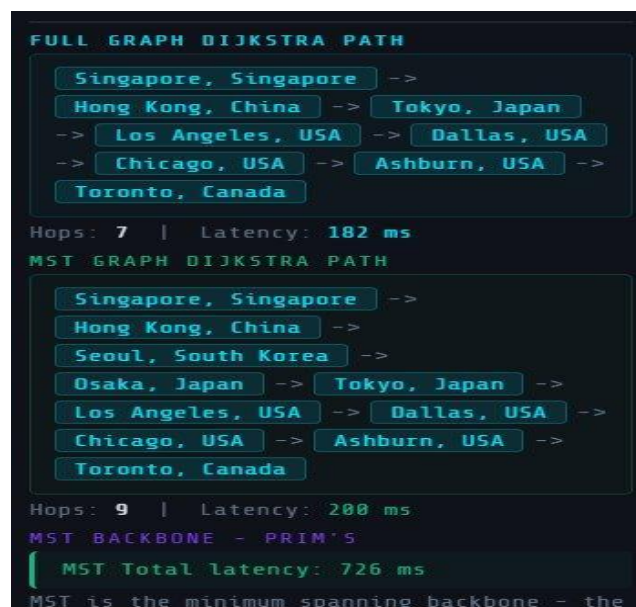


Figure 7: Detailed comparison panel — Full Graph Dijkstra path (7 hops, 182 ms) vs MST Dijkstra path (9 hops, 200 ms); MST Backbone total latency: 726 ms

Key Insight:

- Full Graph Dijkstra (7 hops, 182 ms): Singapore → Hong Kong → Tokyo → Los Angeles → Dallas → Chicago → Ashburn → Toronto
- MST Dijkstra (9 hops, 200 ms): Singapore → Hong Kong → Seoul → Osaka → Tokyo → Los Angeles → Dallas → Chicago → Ashburn → Toronto
- The MST forces routing through its backbone edges, adding 2 hops and 18 ms vs the optimal path

- This demonstrates why ISPs use Dijkstra for packet routing but MST for physical cable infrastructure planning

4.6 GRAPH — Dynamic Network Editor

The GRAPH tab transforms NetRoute into a live network editor. Users can add new nodes (with a custom location name), add edges with both Latency (ms) and Cost (USD) values, and delete existing nodes — all with immediate re-rendering on the D3.js canvas. This makes it possible to simulate custom network topologies beyond the default 30-node dataset.



Figure 8: GRAPH tab — adding a new node (ID: 20, Location: Alwar) and connecting it with an edge from node 3 (Latency: 30 ms, Cost: 343253 USD)

- Add Node: specify Node ID + Location Name → instantly appears on canvas
- Add Edge: specify From ID, To ID, Latency (ms), and Cost (USD)
- Delete Node: remove any node and all its associated edges
- The dual-metric edge system (latency + cost) allows switching filter context for any algorithm

5. Dual Filter System — Latency vs Cost

Every algorithm in NetRoute supports two independent edge-weight metrics, selectable via the FILTER dropdown:

Filter	Edge Weight Used	Real-World Analogy
Latency (ms)	latency_ms field from Data.json	Network delay (ping time); used in OSPF & BGP routing

Cost (USD)	cost_usd field from Data.json	Financial cost of using a link; used in WAN leased-line budgeting & ISP peering agreements
------------	-------------------------------	--

Switching the filter changes the edge weights fed to the C++ algorithm, producing potentially different shortest paths and MSTs. For example, the lowest-latency path may traverse expensive transatlantic cables, while the lowest-cost path may use cheaper but slower regional links — a trade-off that is fundamental to real ISP route selection.

6. System Architecture

6.1 Three-Tier Architecture

Tier	Technology	Responsibility
Frontend	HTML5, CSS3, JS, D3.js	UI rendering, force-directed graph, user interaction, real-time algorithm result display
Backend	Node.js, Express.js	REST API server, child_process spawning of C++ exe, JSON piping, static file serving
Algorithm Engine	C++17 (compiled exe)	Dijkstra, Prim's, Kruskal's, Union-Find; reads Data.json via nlohmann/json; outputs JSON to stdout

6.2 Data Flow

- User selects algorithm parameters and clicks action button in browser
- Frontend sends POST request to Express.js API (e.g. POST /shortest-path)
- Node.js spawns compiled main.exe as a child process with parameters (src, dst, filter)
- C++ reads Data.json, runs the algorithm, and writes JSON result to stdout
- Node.js captures stdout and sends JSON response back to the browser
- D3.js re-renders the graph with highlighted path/MST edges and updates the info panel

6.3 File Structure

```
|— main.cpp           # Main C++ file with CLI and command-line support
|— shortest_path.cpp  # Dijkstra's algorithm implementation
|— mst.cpp            # Prim's and Kruskal's MST algorithms
|— utils.cpp          # Utility functions
|— Data.json          # Network graph data
|— backend.js         # Node.js Express server
|— package.json       # Node.js dependencies
|— frontend/
|   |— index.html     # Main HTML page
```

```

|   └─ style.css           # CSS styling
|   └─ script.js          # JavaScript functionality
└─ README.md              # This file

```

6.4 API Endpoints

Endpoint	Method	Description
/shortest-path	POST	Dijkstra single pair — body: { from, to, filter }
/dijkstra-all	POST	All-nodes shortest path — body: { src, filter }
/mst/prim	POST	Prim's MST — body: { filter }
/mst/kruskal	POST	Kruskal's MST — body: { filter }; filter: 1=latency, 2=cost

7. Build & Execution

7.1 Prerequisites

- C++17 compiler — g++ (Linux/macOS) or MSVC (Windows)
- Node.js >= 14.0.0 and npm
- Modern browser — Chrome, Firefox, or Edge

7.2 Setup Commands

```

# Step 1 — Compile C++ algorithm engine
g++ -std=c++17 -O2 main.cpp shortest_path.cpp mst.cpp utils.cpp -o main.exe

# Step 2 — Install Node.js dependencies
npm install

# Step 3 — Start the backend server
npm start
or: node backend.js

# Step 4 — Open in browser# Navigate to: http://localhost:3000

# Development mode (auto-restart on file change)npm run dev
uses nodemon

```

7.3 CLI Mode (Direct C++ Usage)

```

# Shortest path

./main.exe shortest <from> <to> <filter>

```

```
# Dijkstra all nodes

./main.exe dijkstra_all <src> <filter>


# Prim's MST

./main.exe mst_prim <filter>


# Kruskal's MST

./main.exe mst_kruskal <filter>


# Filter values:

# 1 = latency (ms)

# 2 = cost (USD)
```

8. Network Topology

The simulation runs on a 30-node, 51-edge global network modelled on real internet exchange points and major data centre locations. Each edge carries two weights: latency_ms (propagation delay) and cost_usd (leased-line cost), stored in Data.json.

ID	City	ID	City	ID	City
0	Ashburn, USA	1	London, UK	2	Frankfurt, Germany
3	Tokyo, Japan	4	Singapore	5	Los Angeles, USA
6	Amsterdam, NL	7	Mumbai, India	8	São Paulo, Brazil
9	Sydney, Australia	21	Hong Kong, China	22	Seoul, South Korea
23	Osaka, Japan	24	Jakarta, Indonesia	25	Chennai, India
26	Johannesburg, SA	27	Lagos, Nigeria	28	Santiago, Chile
29	Melbourne, Australia	13	Dallas, USA	15	Toronto, Canada

9. Real-World Applications

The algorithms implemented in NetRoute are not academic exercises — they power real production systems globally. Below are concrete industry applications directly analogous to what NetRoute simulates.

9.1 Internet Routing Protocols

- **OSPF** — OSPF (Open Shortest Path First):
- Used by enterprise routers (Cisco, Juniper) to build per-router forwarding tables using Dijkstra's algorithm. NetRoute models this exact process on every FIND SHORTEST PATH click.
- **BGP** — BGP (Border Gateway Protocol):
- Governs inter-AS routing across the global internet. Path selection logic mirrors shortest-path computation weighted by latency and cost — exactly NetRoute's two filters.

9.2 Telecom Fiber Network Planning

- Telecom operators (Jio, BSNL, AT&T, Vodafone) use MST algorithms to plan the minimum-cost physical fiber cable layout connecting cities
- Kruskal's algorithm sorts all possible cable routes by cost and selects the minimum set that keeps all cities connected — identical to NetRoute's Kruskal's MST with Cost filter
- The MST Backbone total of 726 ms in NetRoute demonstrates that the backbone is not the fastest path — it is the most economical connected structure

9.3 Data Center Networking

- Hyperscalers (Google, Amazon, Meta) use Dijkstra-based routing inside data centers for east-west traffic between servers
- Fat-tree and Clos network topologies use shortest-path routing to minimise latency for distributed microservices
- The 30-node global topology in NetRoute is analogous to a large corporate WAN or cloud provider's regional network

9.4 GPS & Navigation

- Google Maps, Apple Maps, and Waze run Dijkstra's or A* on road-graph networks for every route query
- NetRoute uses the identical algorithm — the only difference is that the nodes are cities and edges are cables, not road intersections and streets
- The dual-filter system mirrors real navigation: fastest route (latency) vs cheapest route (cost/toll)

9.5 Logistics & Supply Chain

- Amazon, FedEx, and DHL use graph algorithms to minimise delivery routes and hub-to-hub infrastructure cost
- MST is used to design distribution networks with minimum total infrastructure investment
- Dijkstra's handles real-time package routing decisions where speed (latency) is prioritised

10. Future Enhancements

NetRoute has strong potential for extension into a full-featured network research and education platform. The following enhancements are proposed:

10.1 Additional Algorithms

- Bellman-Ford: Handle negative edge weights; detect negative cycles — useful for cost-arbitrage simulations
- A* Search: Heuristic-guided pathfinding using geographic coordinates for significantly faster routing on embedded networks
- Floyd-Warshall: All-pairs shortest path with a full $N \times N$ distance matrix visualization
- Max-Flow (Ford-Fulkerson): Model bandwidth capacity constraints and bottleneck detection

10.2 Network Realism

- Dynamic edge weight fluctuation: Simulate real-time congestion events and latency spikes
- Probabilistic link failure: Toggle edges offline and watch the network re-route dynamically
- Bandwidth constraints: Limit flow per edge and show congestion using colour gradients
- Multi-metric optimization: Combine latency and cost into a weighted composite score

10.3 Visualization Improvements

- Step-by-step algorithm animation: Show Dijkstra's relaxation node-by-node with adjustable playback speed
- 3D globe visualization using Three.js or WebGL for a production-grade geographic display
- Side-by-side split view: Run Prim's and Kruskal's on the same canvas simultaneously
- Heatmap overlay: Colour nodes by their distance from the selected source

10.4 Backend & Scalability

- WebSocket support for real-time bidirectional streaming of algorithm state to the frontend
- Docker containerization for one-command deployment on any machine
- Custom graph import: Upload any JSON graph file and run all four algorithms on it
- REST API documentation via Swagger/OpenAPI

11. Algorithm Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Core Data Structure
Dijkstra (Single Pair)	$O((V+E) \log V)$	$O(V + E)$	Adjacency List + Min-Heap
Dijkstra (All Nodes)	$O((V+E) \log V)$	$O(V + E)$	Adjacency List + Min-Heap
Prim's MST	$O(E \log V)$	$O(V + E)$	Priority Queue + Visited Array

Kruskal's MST	$O(E \log E)$	$O(V + E)$	Union-Find DSU + Sorted Edges
---------------	---------------	------------	-------------------------------

For the NetRoute topology: $V = 30$ nodes, $E = 51$ edges. All algorithms run in near-instantaneous time for this scale. The architecture is designed to scale to thousands of nodes without changing the frontend or API layer — only the C++ engine and Data.json need to grow.

12. Conclusion

NetRoute shows how core graph algorithms actually work in real-world network routing. Instead of keeping things theoretical, it turns them into something you can see, interact with, and understand intuitively.

The system combines a fast C++ computation engine with a live, web-based interface, making even complex algorithms feel clear and approachable. It successfully implements four key algorithms — Dijkstra (for both single paths and all nodes), Prim's MST, and Kruskal's MST — on a global network of 30 nodes and 51 edges, using two realistic metrics: latency and cost.

One of its most interesting features is the comparison between MST and Dijkstra. It clearly shows something that's often missed in classrooms: building the most efficient overall network (MST) is very different from finding the best path between two points (shortest path). Seeing this difference visually makes the concept much easier to grasp.

With features like a dynamic graph editor, dual filtering options, and a clean three-layer architecture, NetRoute is not just a finished project but a flexible platform that can grow with future improvements.

At its core, NetRoute connects theory with reality. The same Dijkstra's algorithm running inside it is also quietly working behind the scenes on the internet, helping route data across the world every second.
